

■ Layout

- HTML 요소 간의 위치 및 부모 컨테이너와 비례한 위치 등을 제어
- 사용 속성
 - display (block / inline / inline-block / none)
: 요소가 보여지는 방식 (한줄 / 내용만큼 / 숨김)
 - position (static / relative / absolute / fixed / sticky)
: 요소의 위치와 관련된 배치 방식 (기본 / 상대 / 절대 등)
 - grid : 표 형태 + 위치 및 크기 명시
 - flexbox : 가변 배치 형태, 유연한 반응형 레이아웃 구조 지원

[https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS layout/Introduction](https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Introduction)

■ Layout - display

- 부모 크기 / 내용물 크기 / 숨기기

- 속성 값

- **block** : 부모의 크기대로 맞춤, 한 줄 전체 (크기 조절 가능)

- **inline** : 내용물의 크기만큼 (크기 조절 불가)

- **inline-block** : 내용물의 크기만큼 (크기 조절 가능)

- **none** : 숨기기

```
<style>
  div { border: 1px solid red; }
  div:nth-of-type(1) { display: block; }
  div:nth-of-type(2) { display: inline; width: 50vw; height: 50px;}
  div:nth-of-type(3) { display: inline-block; width: 50vw; height: 50px;}
</style>
```

```
<body>
  <div>block 제목</div>
  <span>block 내용</span> <hr>
  <div>inline</div>
  <span>inline 내용</span> <hr>
  <div>inline-block</div>
  <span>inline-block 내용</span>
</body>
```

block 제목

block 내용

inline inline 내용

inline-block

inline-block 내용

■ Layout - display

● none : 숨기기

```
<style>
  body { user-select: none; }
  input { display: none; }
  div {
    display: none; background-color: lightpink;
    width: 90vw; height: 30vh;
  }
  input:checked + label { background-color: purple; }
  input:nth-of-type(1):checked ~ div:nth-of-type(1) { display: block; }
  input:nth-of-type(2):checked ~ div:nth-of-type(2) { display: block; }
  input:nth-of-type(3):checked ~ div:nth-of-type(3) { display: block; }
</style>
```

```
<body>
  <input type="radio" id='a' name='menu'>
  <label for="a">A 메뉴</label>
  <input type="radio" id='b' name='menu'>
  <label for="b">B 메뉴</label>
  <input type="radio" id='c' name='menu'>
  <label for="c">C 메뉴</label>
  <div>A 내용</div>
  <div>B 내용</div>
  <div>C 내용</div>
</body>
```

A 메뉴 B 메뉴 C 메뉴

A 메뉴 B 메뉴 C 메뉴
A 내용

A 메뉴 B 메뉴 C 메뉴
C 내용

■ 연습문제

● 결과와 같이 보여지도록 코드 작성하기

- 각 항목 선택 시 입력창 보이기
- 기본 코드

```
<style>

</style>

<body>
  <input type='checkbox' id='a'>
  <label for='a'>정보 1</label>
  <textarea name='info_a'></textarea><br>
  <input type='checkbox' id='b'>
  <label for='b'>정보 2</label>
  <textarea name='info_b'></textarea><br>
  <input type='checkbox' id='c'>
  <label for='c'>정보 3</label>
  <textarea name='info_c'></textarea>
</body>
```

결과

- ☐ 정보 1
- ☐ 정보 2
- ☐ 정보 3



- ☒ 정보 1
- ☒ 정보 2
- ☒ 정보 3

■ 연습문제

- 결과와 같이 보여지도록 코드 작성하기
 - 기본 코드

```
<style>

</style>

<body>
  <div id="content">

    <div>
      <h1>공지사항</h1>
      <ul><li>게시물_01</li>
      <li>게시물_02</li></ul>
    </div>

    <!-- 위 공지사항 코드 2번 반복 -->

  </div>
</body>
```

결과

공지사항

- 게시물_01
- 게시물_02

공지사항

- 게시물_01
- 게시물_02

공지사항

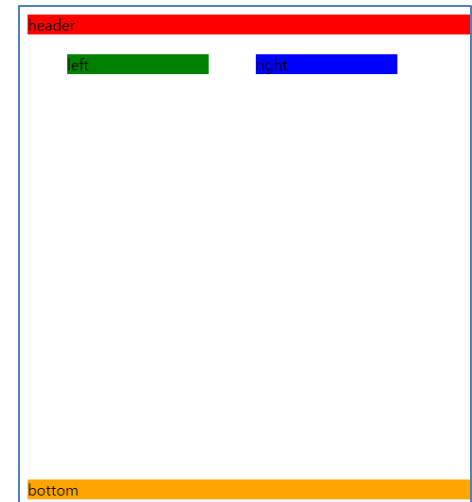
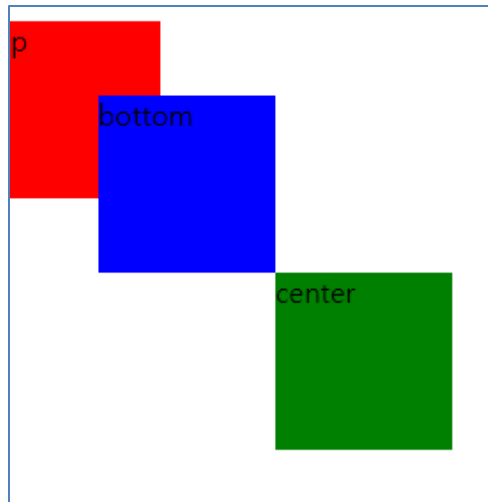
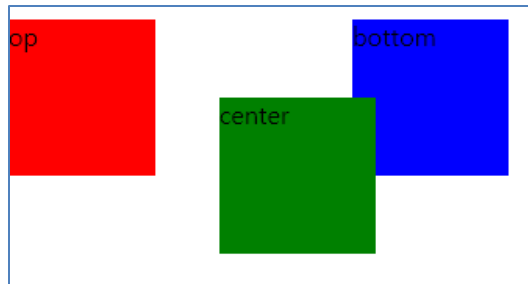
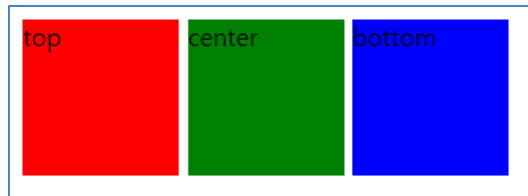
- 게시물_01
- 게시물_02

■ Layout - position

● 요소가 배치되는 방식

● 속성 값

- static : 기본 값, 흐름대로 배치
- relative : static 의 배치 방식에 추가로 위치 이동 가능
- absolute : **HTML 문서 기준** 절대 좌표를 사용하여 배치
- fixed : 사용자의 **브라우저 기준** 절대 좌표를 사용하여 배치
- sticky : static 의 배치 방식에 추가로 fixed와 같이 위치 고정



■ Layout - position

● static : 기본 값, 흐름대로 배치

```
<style>
  body {
    width: 500px;
    margin: 0 auto;
  }

  p {
    background-color: rgb(207, 232, 220);
    border: 2px solid rgb(79, 185, 227);
    padding: 10px;
    margin: 10px;
    border-radius: 5px;
  }
</style>

<body>
  <h1>static 적용</h1>
  <p>static</p>
  <p>static</p>
  <p>static</p>
</body>
```

static 적용

static

static

static

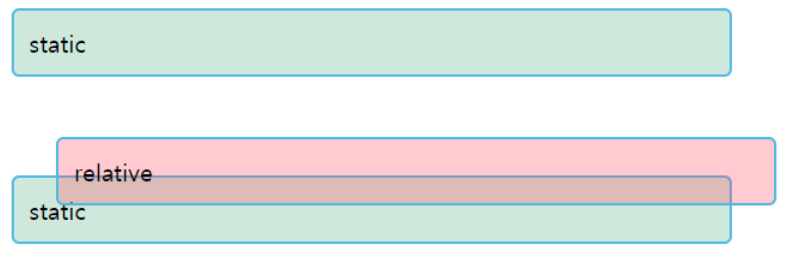
■ Layout - position

- relative : static 의 배치 방식에 추가로 위치 이동 가능

```
<style>
  body { width: 500px; margin: 0 auto; }
  p {
    background-color: rgb(207, 232, 220);
    border: 2px solid rgb(79, 185, 227);
    padding: 10px; margin: 10px;
    border-radius: 5px;
  }
  .rel {
    position: relative;
    top: 30px; left: 30px;
    background-color: rgba(255,84,104,.3)
  }
</style>
```

```
<body>
  <h1>relative 적용</h1>
  <p>static</p>
  <p class='rel'>relative</p>
  <p>static</p>
</body>
```

relative 적용



left / top / right / bottom 속성으로 위치 이동 가능

위치가 바뀌어도 다음 요소가 배치되는 기준은 실제 위치

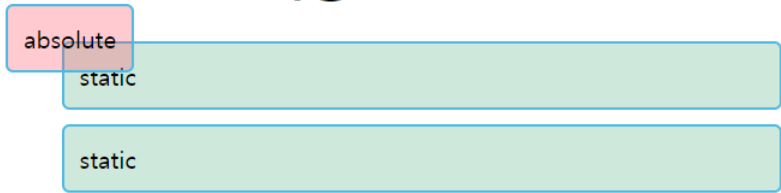
■ Layout - position

● absolute : HTML 문서 기준 절대 좌표를 사용하여 배치

```
<style>
  body { width: 500px; margin: 0 auto; }
  p {
    background-color: rgb(207, 232, 220);
    border: 2px solid rgb(79, 185, 227);
    padding: 10px; margin: 10px;
    border-radius: 5px;
  }
  .abs {
    position: absolute;
    top: 30px; left: 30px;
    background-color: rgba(255,84,104,.3)
  }
</style>

<body>
  <h1>absolute 적용</h1>
  <p>static</p>
  <p class='abs'>absolute</p>
  <p>static</p>
</body>
```

absolute 적용



absolute

static

static

left / top / right / bottom 속성으로 위치 이동 가능

기본 배치 흐름과는 상관없이 원하는 위치에 요소 배치 가능 (HTML 문서 기준)

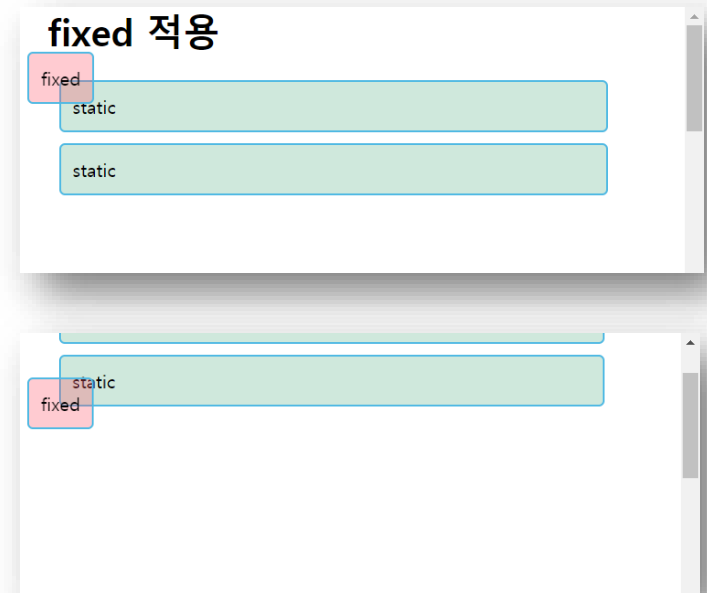
위치 이동의 기준은 static이 아닌 부모 요소

■ Layout - position

- fixed : 사용자의 브라우저 기준 절대 좌표를 사용하여 배치

```
<style>
  body { width: 500px; margin: 0 auto; }
  p {
    background-color: rgb(207, 232, 220);
    border: 2px solid rgb(79, 185, 227);
    padding: 10px; margin: 10px;
    border-radius: 5px;
  }
  .fix {
    position: fixed; top: 30px; left: 30px;
    background-color: rgba(255,84,104,.3)
  }
</style>

<body>
  <h1>fixed 적용</h1>
  <p>static</p>
  <p class='fix'>fixed</p>
  <p>static</p>
  <!-- <br> * 100 -->
</body>
```



left / top / right / bottom 속성으로 위치 이동 가능

기본 배치 흐름과는 상관없이 원하는 위치에 요소 배치 가능 (브라우저-Viewport 기준)

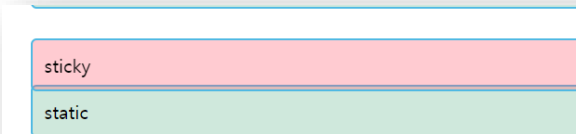
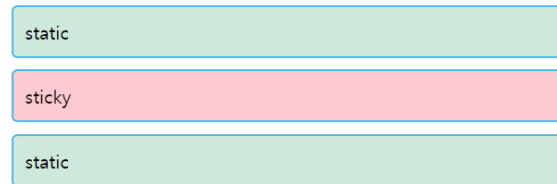
■ Layout - position

- sticky : static 의 배치 방식에 추가로 fixed와 같이 위치 고정

```
<style>
  body { width: 500px; margin: 0 auto; }
  p {
    background-color: rgb(207, 232, 220);
    border: 2px solid rgb(79, 185, 227);
    padding: 10px; margin: 10px;
    border-radius: 5px;
  }
  .sticky {
    position: sticky; top: 30px; left: 30px;
    background-color: rgba(255,84,104,.3)
  }
</style>

<body>
  <h1>sticky 적용</h1>
  <p>static</p>
  <p class='sticky'>sticky</p>
  <p>static</p>
  <!-- <br> * 100 -->
</body>
```

sticky 적용



left / top / right / bottom 속성으로 위치 이동 가능

기본 배치 흐름과는 상관없이 원하는 위치에 요소 배치 가능 (브라우저-ViewPort 기준)

■ 연습문제

● 결과와 같이 보여지도록 코드 작성하기

- 스크롤바를 이동해도 상단의 header 고정
- 이미지 : ggoreb.com/images/food/a.png ~ j.png (10개)
- 기본 코드

```
<style>
  body { margin: 0; }
  .img { width: 40vw; }
</style>
<body>
  <div class="header">
    <h1>음식사진</h1>
  </div>
  <div class="container">
    
    <!-- 이미지 9개 추가 -->
  </div>
</body>
```

결과



■ Layout - grid

- 표 형태의 2차원 레이아웃을 만드는 방식
- 행과 열을 쉽게 조절할 수 있어서 복잡한 배치에 유리함
 - 카드형 UI, 게시판, 대시보드 등 반복형 구조에 많이 사용
- 주요 속성

속성	설명
display: grid	grid 레이아웃 사용 선언
grid-template-columns	열의 개수와 너비 지정
grid-template-rows	행의 개수와 너비 지정
gap	셀 간 간격 지정 (flexbox 등 다른 레이아웃에 사용 가능)
<i>grid-gap</i>	<i>셀 간 간격 지정 (grid 전용)</i>
grid-column	열의 위치 또는 범위 지정
grid-row	행의 위치 또는 범위 지정

- 단위
 - px, %
 - auto : 내용 크기만큼 자동 조절
 - fr : 남은 공간을 나눠쓰는 단위 (fraction)

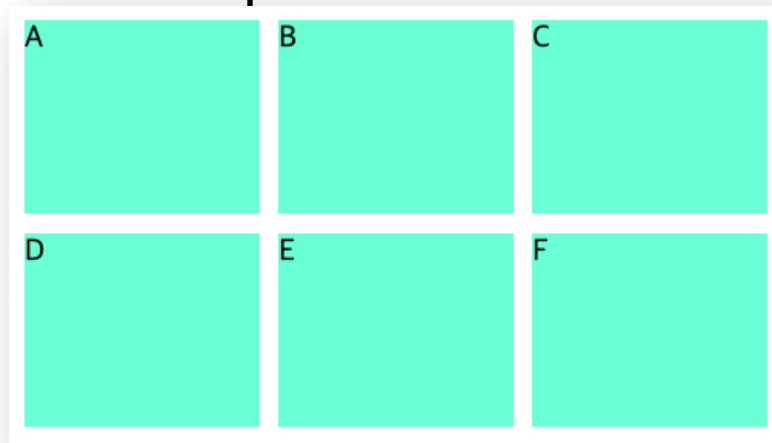
■ Layout - grid

● 기본 사용 형태

```
<style>
  .wrapper {
    display: grid; gap: 10px;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: 100px 100px;
  }

  div > div {
    background-color: aquamarine;
  }
</style>

<body>
  <div class="wrapper">
    <div>A</div> <div>B</div> <div>C</div>
    <div>D</div> <div>E</div> <div>F</div>
  </div>
</body>
```



■ Layout - grid

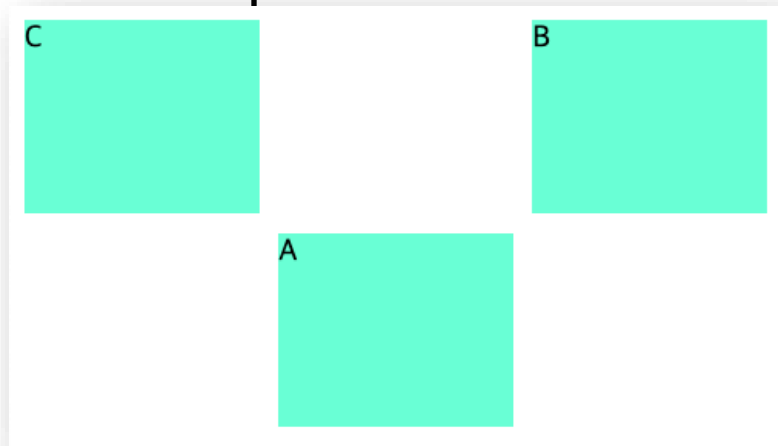
● 셀의 위치 지정

```
<style>
  .wrapper {
    display: grid; gap: 10px;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: 100px 100px;
  }

  div > div {
    background-color: aquamarine;
  }

  .box1 { grid-row: 2; grid-column: 2; }
  .box2 { grid-row: 1; grid-column: 3; }
  .box3 { grid-row: 1; grid-column: 1; }
</style>

<body>
  <div class="wrapper">
    <div class="box1">A</div>
    <div class="box2">B</div>
    <div class="box3">C</div>
  </div>
</body>
```



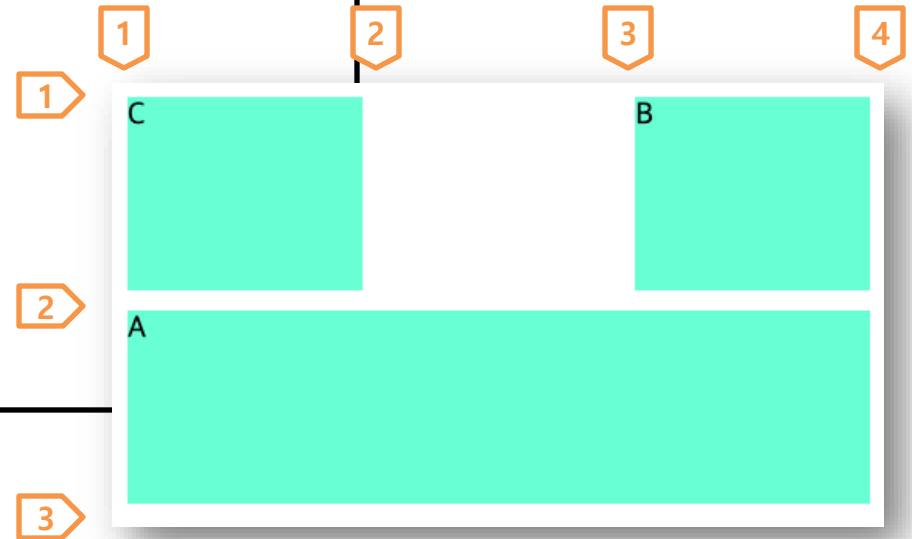
■ Layout - grid

● 셀의 범위 지정

```
<style>
...

.box1 { grid-row: 2; grid-column: 1 / 4; }
.box2 { grid-row: 1; grid-column: 3; }
.box3 { grid-row: 1; grid-column: 1; }
</style>

<body>
  <div class="wrapper">
    <div class="box1">A</div>
    <div class="box2">B</div>
    <div class="box3">C</div>
  </div>
</body>
```



■ 연습문제

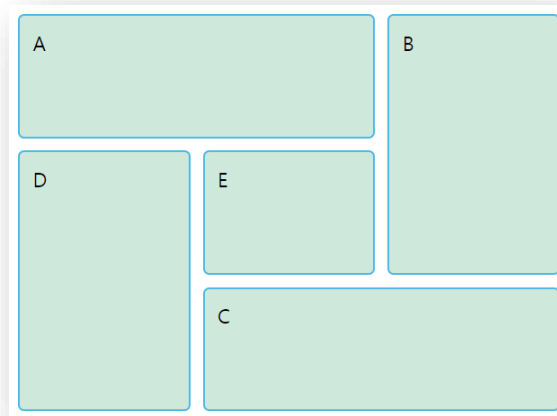
● 결과와 같이 보여지도록 코드 작성하기

- 3행 4열의 Grid Layout, 5개 요소
- 기본 코드

```
<style>
  div > div {
    background-color: rgb(207, 232, 220);
    border: 2px solid rgb(79, 185, 227);
    padding: 10px;
    border-radius: 5px;
  }
  .wrapper {
    display: grid; grid-gap: 10px;
  }
</style>

<body>
  <div class="wrapper">
    <div>A</div>
    <div>B</div>
    <div>C</div>
    <div>D</div>
    <div>E</div>
  </div>
</body>
```

결과



■ 연습문제

● 결과와 같이 보여지도록 코드 작성하기 (1 / 2)

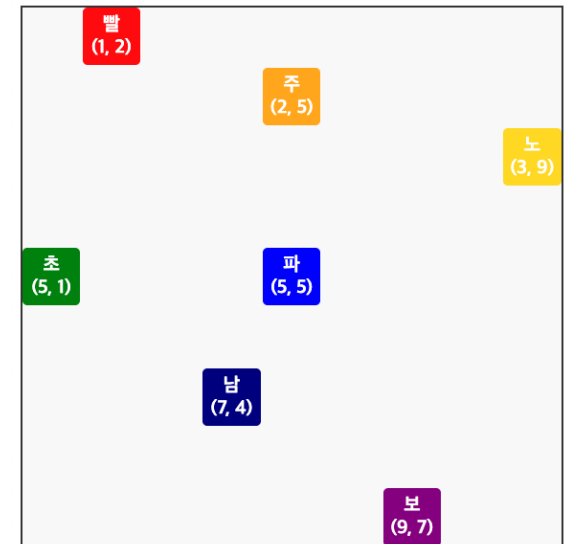
- 9행 9열의 Grid Layout을 구성
- 빨주노초파남보 색상 박스를 아래 좌표에 맞게 배치
- 기본 코드

```
<body>
  <style>
    .board {
      display: grid; gap: 2px;
      width: 450px; aspect-ratio: 1 / 1;
      grid-template-columns: repeat(9, 1fr);
      grid-template-rows: repeat(9, 1fr);
      border: 2px solid #333;
      background-color: #f8f8f8;
      text-align: center;
    }

    .color-box {
      display: flex;
      justify-content: center;
      align-items: center;
      color: white;
      font-weight: bold;
      border-radius: 4px;
    }
  </style>

```

결과



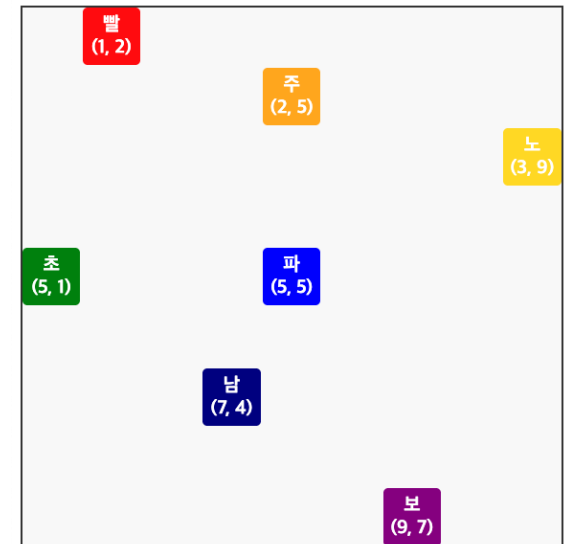
■ 연습문제

● 결과와 같이 보여지도록 코드 작성하기 (2 / 2)

- 9행 9열의 Grid Layout을 구성
- 빨주노초파남보 색상 박스를 아래 좌표에 맞게 배치
- 기본 코드

```
<div class="board">  
  <div class="color-box red">빨<br>(1, 2)</div>  
  <div class="color-box orange">주<br>(2, 5)</div>  
  <div class="color-box yellow">노<br>(3, 9)</div>  
  <div class="color-box green">초<br>(5, 1)</div>  
  <div class="color-box blue">파<br>(5, 5)</div>  
  <div class="color-box navy">남<br>(7, 4)</div>  
  <div class="color-box purple">보<br>(9, 7)</div>  
</div>  
</body>
```

결과

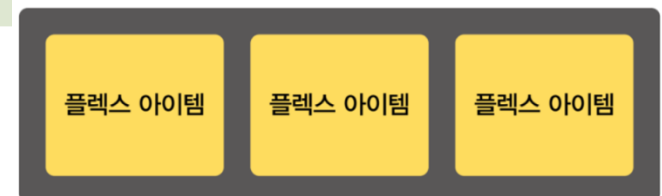


■ Layout - flexbox

- 가변 배치 형태, 유연한 반응형 레이아웃 구조 지원
- 실무에서 가장 많이 사용되는 레이아웃 중 하나
 - 요소를 가로 또는 세로 방향으로 배치
 - 요소 간 간격, 정렬, 순서 등을 손쉽게 조절 가능
- 사용 속성

Container	Item
display	flex
flex-direction	flex-grow
justify-content	flex-shrink
align-items	flex-basis
flex-wrap	order
flex-flow	align-self
align-content	

플렉스 컨테이너



■ Layout - flexbox

● 기본 적용 (display: flex)

```
<style>
  .flex-container {
    display: flex;
    background-color: lightgrey;
  }

  .flex-item {
    background-color: lightcyan;
    border: 1px solid red;
  }
</style>
```

```
<body>
  <div class="flex-container">
    <div class="flex-item">1</div>
    <div class="flex-item">2</div>
    <div class="flex-item">3</div>
  </div>
</body>
```

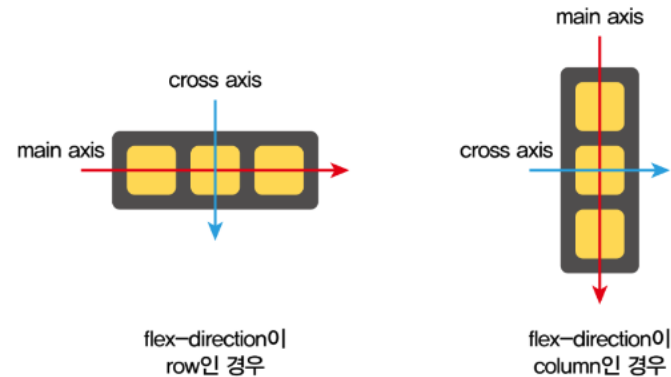
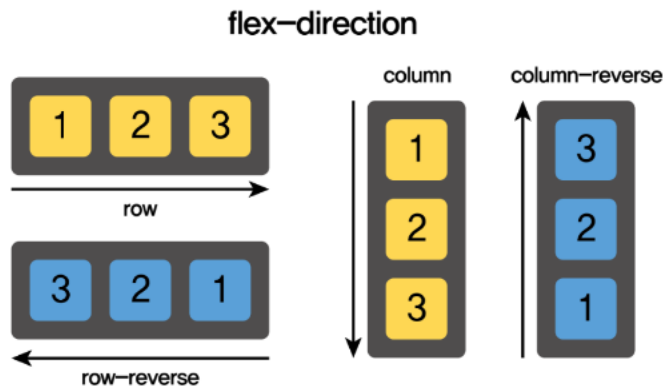


■ Layout - flexbox

● flex-direction : 요소 배치 방향 지정

flex-direction

- row: 기본값이며 아이টে를 행^{row}을 따라 가로 순서대로 왼쪽부터 배치합니다.
- column: 아이টে를 열^{column}을 따라 세로 순서대로 위쪽부터 배치합니다.
- row-reverse: 아이টে를 행^{row}의 역^{reverse}방향으로 오른쪽부터 배치합니다.
- column-reverse: 아이টে를 열^{column}의 역^{reverse}방향으로 아래쪽부터 배치합니다.



■ Layout - flexbox

● flex-direction : 요소 배치 방향 지정

```
<style>
  .flex-container {
    display: flex;
    background-color: lightgrey;
    flex-direction: column;
  }

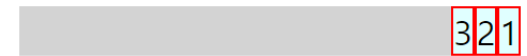
  .flex-item {
    background-color: lightcyan;
    border: 1px solid red;
  }
</style>

<body>
  <div class="flex-container">
    <div class="flex-item">1</div>
    <div class="flex-item">2</div>
    <div class="flex-item">3</div>
  </div>
</body>
```

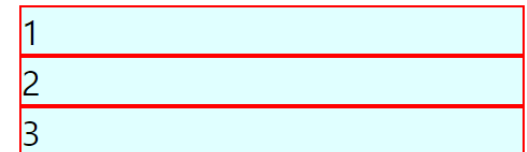
row (기본)



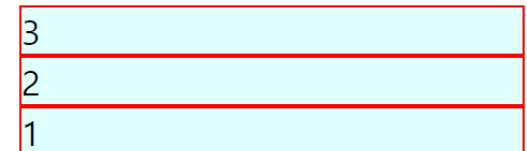
row-reverse



column



column-reverse



■ Layout - flexbox

● justify-content (요소 정렬 - main axis 기준)

justify-content

- `flex-start`: main axis의 시작 지점으로 아이টে을 맞춥니다.
- `center`: main axis의 중앙으로 아이টে을 맞춥니다.
- `flex-end`: main axis의 끝 지점으로 아이টে을 맞춥니다.
- `space-between`: main axis를 기준으로 첫 아이টে은 시작 지점에 맞추고 마지막 아이টে은 끝 지점에 맞추며, 중간에 있는 아이টে들 사이 ^{between}의 간격 ^{space}이 일정하게 되도록 맞춥니다.
- `space-around`: main axis를 기준으로 각 아이টে의 주변 ^{around} 간격 ^{space}을 동일하게 맞춥니다. 여기에 서 주변의 의미는 아이টে의 시작과 끝을 의미합니다.

justify-content

flex-start



space-between



center



space-around



flex-end

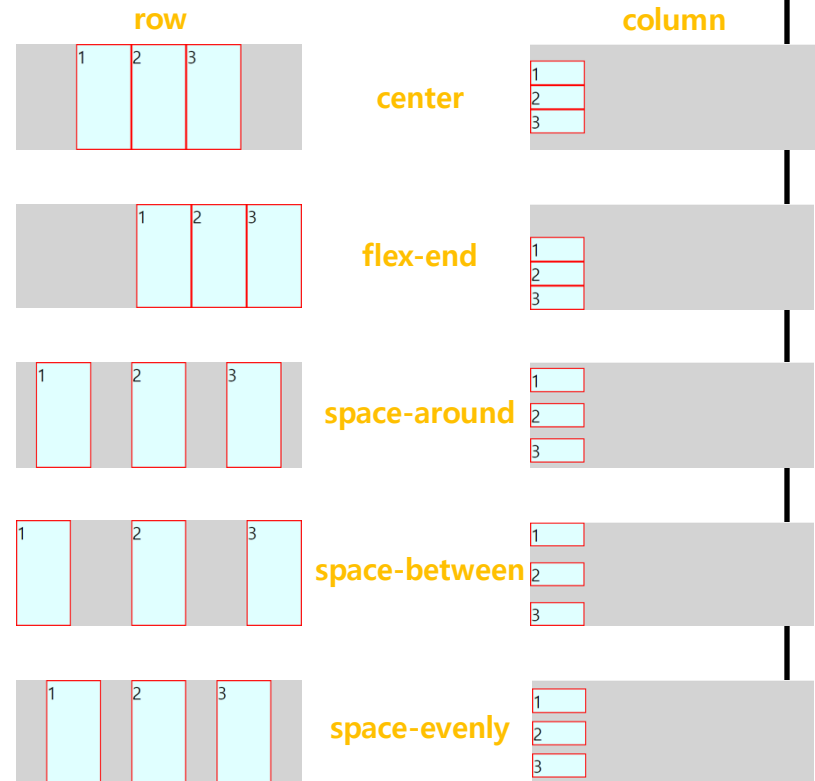


■ Layout - flexbox

● justify-content (요소 정렬 - main axis 기준)

```
<style>
  .flex-container {
    display: flex;
    background-color: lightgrey;
    flex-direction: row;
    justify-content: center;
  }
  .flex-item {
    width: 50px; height: 50px;
    background-color: lightcyan;
    border: 1px solid red;
  }
</style>
```

```
<body>
  <div class="flex-container">
    <div class="flex-item">1</div>
    <div class="flex-item">2</div>
    <div class="flex-item">3</div>
  </div>
</body>
```



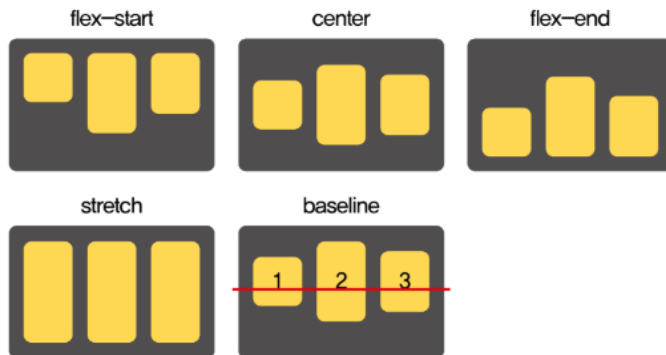
■ Layout - flexbox

● align-items (반대축의 요소 정렬)

align-items

- stretch: 기본값으로써 아이템을 늘려서 stretch 컨테이너를 가득 채웁니다.
- flex-start: cross axis의 시작 지점으로 아이템을 정렬합니다.
- center: cross axis의 중앙으로 아이템을 정렬합니다.
- flex-end: cross axis의 끝 지점으로 아이템을 정렬합니다.
- baseline: 아이템을 baseline 기준으로 정렬합니다.

align-items

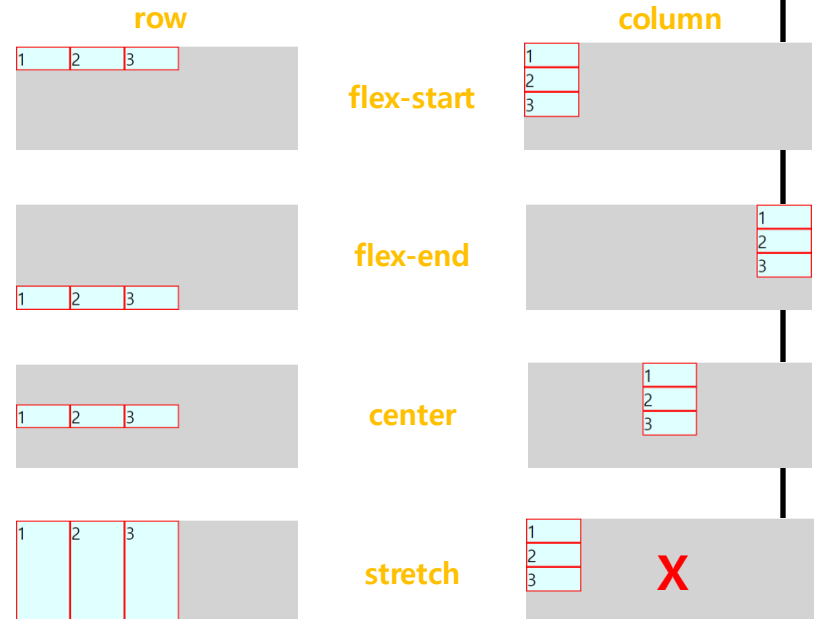


■ Layout - flexbox

● align-items (반대축의 요소 정렬)

```
<style>
  .flex-container {
    display: flex;
    background-color: lightgrey;
    flex-direction: row;
    align-items: center;
    height: 100px;
  }
  .flex-item {
    width: 50px;
    background-color: lightcyan;
    border: 1px solid red;
  }
</style>
```

```
<body>
  <div class="flex-container">
    <div class="flex-item">1</div>
    <div class="flex-item">2</div>
    <div class="flex-item">3</div>
  </div>
</body>
```



■ 연습문제

● 결과와 같이 보여지도록 코드 작성하기

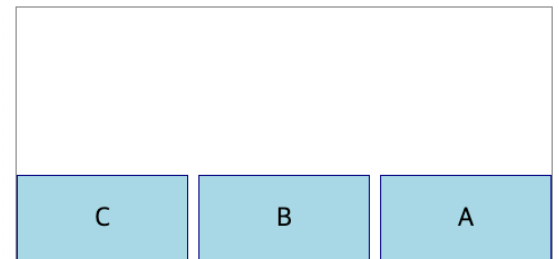
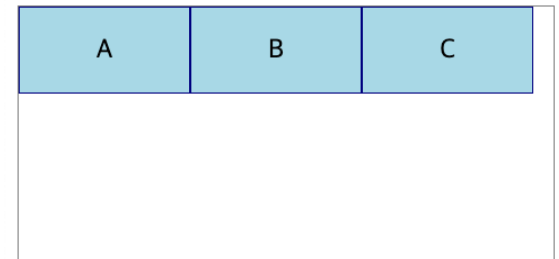
- 3개의 박스를 가로로 배치
- flex-direction, justify-content, align-items 사용
- 기본 코드

```
<style>
  .container {
    display: flex;
    height: 150px;
    border: 1px solid gray;
  }

  .container > div {
    width: 100px;
    height: 50px;
    background-color: lightblue;
    text-align: center;
    line-height: 50px;
    border: 1px solid navy;
  }
</style>

<body>
  <div class="container">
    <div>A</div> <div>B</div> <div>C</div>
  </div>
</body>
```

결과

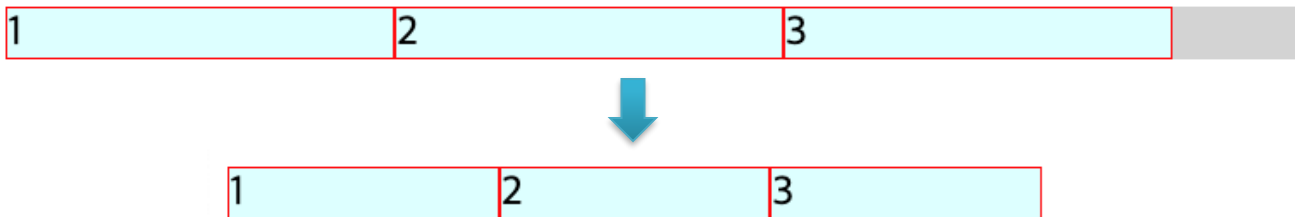


■ Layout - flexbox

● flex-wrap (요소 감싸기)

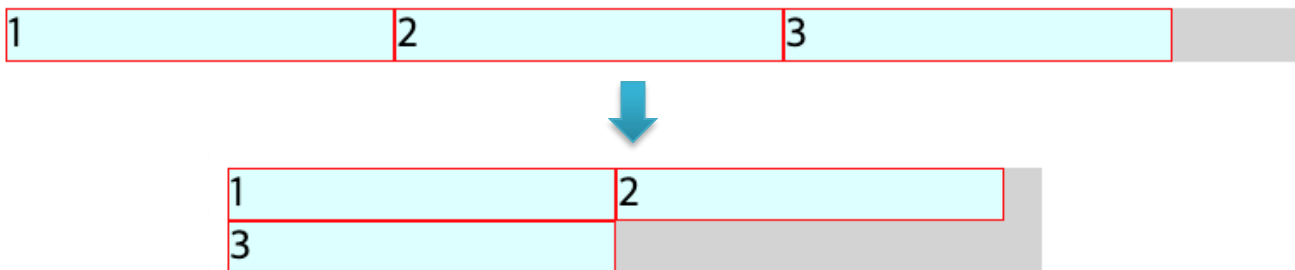
- nowrap (기본)

- 한 줄에 유지
- 요소의 크기를 자동으로 줄여서라도 한줄에 맞춤



- wrap

- 공간이 부족하면 줄바꿈
- 요소의 크기를 줄이지 않고 다음 줄로 넘김



■ Layout - flexbox

● flex-wrap (요소 감싸기)

```
<style>
  .flex-container {
    display: flex;
    background-color: lightgrey;
    flex-wrap: wrap;
  }
  .flex-item {
    width: 200px;
    background-color: lightcyan;
    border: 1px solid red;
  }
</style>

<body>
  <div class="flex-container">
    <div class="flex-item">1</div>
    <div class="flex-item">2</div>
    <div class="flex-item">3</div>
  </div>
</body>
```

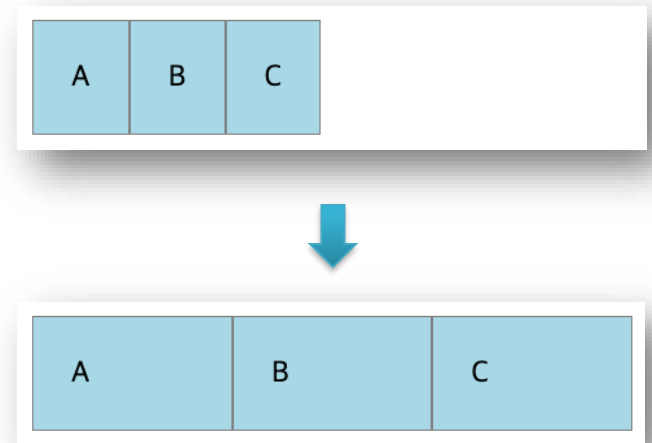
■ Layout - flexbox

● flex (item 크기 비율 조절)

```
<style>
  .flex-container {
    display: flex;
  }

  .flex-item {
    flex: 1;
    padding: 20px;
    border: 1px solid gray;
    background-color: lightblue;
  }
</style>
```

```
<body>
  <div class="flex-container">
    <div class="flex-item">A</div>
    <div class="flex-item">B</div>
    <div class="flex-item">C</div>
  </div>
</body>
```



■ 연습문제

● 결과와 같이 보여지도록 코드 작성하기

- 3개의 박스 중 가운데 박스만 넓게 표시 (1:2:1)
- flex-direction, justify-content, align-items 사용
- 기본 코드

```
<style>
  .container {
    display: flex;
    height: 50vh; gap: 10px;
  }

  .container>div {
    height: 80px; text-align: center;
    line-height: 80px; background-color: pink;
    border: 1px solid red;
  }
</style>

<body>
  <div class="container">
    <div>좌</div> <div>중앙</div> <div>우</div>
  </div>
</body>
```

결과

